

What are the core components in React Native?

Core components are fundamental building blocks of a React Native app. They are pre-built components provided by React Native that are rendered to native UI elements on iOS and Android.

- They are built-in
- They work on both iOS and Android
- They render to native components.
- Must be imported from 'react-native' before usage.
- ★ Most of components have been discussed previously.

Syntax and Purpose of components

`<View>` Component

```
import { View } from 'react-native';
```

```
<View style={{ flex: 1, backgroundColor: 'blue' }} />
```

```
< // other components />
```

```
</View>
```

- Acts like a container (div)
- Used for layout and positioning
- Supports flexbox layout
- Can nest other components
- Can't directly display text, must use `<Text>`

Common Use Cases:

- Container for entire screen
- Row or column layout
- Grouping related comp
- Creating colored box
- Spacing and padding containers.

<Text> Component

```
import {Text} from 'react-native';
```

```
<Text style={{ fontSize: 20, color: 'red',  
fontWeight: 'bold' }} >
```

Hello World!

```
</Text>
```

- Can nest <text> inside <text>
- Supports various text styles

Common Use Cases:

- Heading
- Paragraphs
- Labels
- Any text display

<Button> Component

```
import {Button} from 'react-native';
```

```
<Button
```

```
  title = 'Submit'
```

```
  onPress = {() => alert('Submitted')}}
```

```
  color = 'blue'
```

→ Interactive button that responds on touch.

Note: <Button> has limited styling. For more custom buttons, use <TouchableOpacity> or <Pressable>

Common Props

Props are like attributes that customize components.

Prop	Type	Purpose
style	Object/Array	Apply Styles
onLayout	Function	Triggered when layout changes

For <View> ↗

For <Text>

Prop	Type	Purpose
Style	Object/Array	Apply text styles
numberOfLines	Number	Limit text lines
onPress	Function	Make text touchable

<Button>

Prop	Type	Purpose
title	String	ButtonText
onPress	Function	Click handler
color	String	Button color
disabled	Boolean	Disable Button

Inline - Styling

```
<View style = {{{ padding: 20}}} >
```

```
// other comp
```

```
</View>
```

Pros

- Quick & easy for small styles
- Good for dynamic styles
- No need for Style Sheet • create

Cons

- Creates new on every render (less performant)
- Harder to reuse
- Can make JSX messy

StyleSheet Styling

```
import { StyleSheet } from 'react-native';
```

```
<View style = { style.container }>
```

```
  <Text style = { style.text }>StyleSheet </Text>
```

```
</View>
```

```
const styles = StyleSheet.create({
```

```
  container: {
```

```
    backgroundColor: 'blue',
```

```
    padding: 20,
```

```
  },
```

```
  text: {
```

```
    fontSize: 18,
```

```
    color: 'white',
```

```
  }
```

```
});
```

Pros

- Better Performance
- Reusable styles
- Cleaner JSX
- Easier to maintain
- validation of style properties

Cons

- Extra code
- Seperate from JSX

Simple Layout

App

└ View

└ Text

└ Button

```
import React from 'react';
import { View, Text, Button, StyleSheet }
  from 'react-native';
```

```
export default function App() {
  const handlePress = () => {
    alert('Button was pressed')
  };
  return (
```

```
    <View style={styles.container}>
```

```
      <Text style={style.title}>
```

```
        Welcome to React Native </Text>
```

```
      <Text style={styles.subtitle}>
```

```
        This is simple Layout </Text>
```

```
      <Button
```

```
        title = 'Press Me'
```

```
        onPress = {handlePress}
      </Button>
```

```
    </View>
```



```
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    justifyContent: 'center',  
    alignItems: 'center',  
    backgroundColor: 'gray',  
    padding: 20,  
  },  
  title: {  
    fontSize: 24,  
    fontWeight: 'bold',  
    color: '#333',  
    marginBottom: 10,  
  },  
  subtitle: {  
    fontSize: 16,  
    color: '#666',  
    marginBottom: 20,  
  },  
});
```